

Continuous-Time Recurrent Reinforcement Learning for USV Navigation in Dynamic Maritime Environments

Le Tuan Minh Trinh¹ SN: 23018740

¹ Department of Mechanical Engineering, University College London.

May 8, 2026

Abstract

Navigating Unmanned Surface Vehicles (USVs) in maritime environments requires collision avoidance under time-varying currents, moving obstacles, partial observations, and limited on-board computation. Existing Reinforcement Learning (RL) approaches for USV navigation often assume static or weakly dynamic environments, limiting robustness to dynamic obstacle motion. This project studies USV navigation in a dynamic Rankine vortex-flow simulator with both static and dynamic obstacles, using a limited-range LiDAR sensor representation augmented by engineered direction and clearance features. The task is formulated as a Partially Observable Markov Decision Process (POMDP), and a recurrent policy based on Closed-form Continuous-Time (CfC) neural networks is trained with Proximal Policy Optimization (PPO) under a four-stage timestep-scheduled curriculum. The proposed policy uses a multi-branch feature extractor to separate kinematic, perception, and dynamic-awareness inputs before recurrent belief updating. In simulation, CfC+PPO achieves a 6.2% success rate (SR) improvement over LSTM+PPO in Static evaluation and a 6.4% SR improvement over GRU+PPO in Dynamic evaluation, while reducing collision rate (CR) and learned-policy control effort. Ablations show that recurrence improves robustness to cumulative hidden drift, while engineered spatial summaries are essential for reliable local collision avoidance. The results provide promising simulation-based evidence for a lightweight hybrid policy for adaptive USV navigation in dynamic maritime environments.

Keywords: Unmanned surface vehicles, deep reinforcement learning, partially observable Markov decision process, recurrent neural networks, liquid neural networks.

1 Introduction

Safe navigation and collision avoidance in unknown, dynamic environments represent a core challenge for autonomous Unmanned Surface Vehicles (USVs) (1). Unlike terrestrial road networks or aerial navigation settings, maritime domains are weakly structured and exhibit stochastic behaviour. Ocean currents continuously alter effective heading and speed, while obstacles move with the flow. Furthermore, sparse LiDAR sensor returns only provide a partial geometric snapshot of the surrounding environment at each time step. Therefore, to operate safely, a USV must continuously solve a dynamic decision-making problem: adapting to its own altered motion and avoiding moving obstacles, while operating within the stringent computational and power constraints of embedded marine hardware (2).

Over the past few years, Deep Reinforcement Learning (DRL) has been established as a viable control paradigm for USV navigation (3; 4). However, a critical limitation persists: many existing methods rely on static environmental assumptions or treat ocean-current effects as unknown disturbances, rather than explicitly

modelling their spatio-temporal influence on the vehicle and surrounding obstacles (5; 6). Recent research has begun to tackle these shortcomings by improving policy architectures and input representations. For instance, Convolutional Neural Networks (CNNs) have been integrated with DRL to process multi-scale grid-map representations of vessel encounters, effectively treating dynamic obstacle avoidance as a visual recognition task (7). Other advanced approaches utilise spatio-temporal attention mechanisms to model multi-agent interactions (8), and distributional RL to explicitly capture return uncertainty (9; 10). However, these methods either increase embedded computational cost or are often evaluated in simplified environments. This leaves a gap for lightweight temporal policies that can handle flow-driven obstacle motion under limited onboard sensing.

To address these challenges, this project develops a learning framework built around a dynamic marine simulator and a lightweight recurrent policy. The dynamic simulator features Rankine vortex-induced currents to govern both the flow field and the motion of surrounding obstacles, exposing the agent to flow-driven dynamics. Because the agent relies on noisy, limited-range

observations, the task is modelled as a Partially Observable Markov Decision Process (POMDP). Under these constrained conditions, instantaneous sensor snapshots are insufficient to capture the velocities of moving obstacles or the invisible drift of ocean currents; the agent must therefore rely on a recurrent memory mechanism to integrate past observations over time and infer these latent dynamics. To navigate this setting, the project employs a recurrent policy based on Closed-form Continuous-time (CfC) neural networks (11) combined with Proximal Policy Optimization (PPO) (12) trained using a curriculum learning approach (13). Unlike traditional discrete-time models like Long Short-term Memory (LSTM) (14), which can struggle to map multi-scale environmental changes, the CfC backbone is derived from ordinary differential equations (ODEs). By parameterising hidden-state evolution in continuous time, the CfC architecture is structurally well matched to the sampled, time-varying dynamics used in this simulator. The main contributions of the work are summarised as follows:

- A dynamic marine simulator featuring Rankine vortex currents and flow-driven obstacle motion, enabling controlled evaluation of USV navigation under partial observability (Section 3).
- A CfC-based recurrent policy with multi-branch feature extraction that outperforms discrete-time recurrent baselines by up to 7.4% in success rate and 40% in relative collision reduction, at 45% lower inference latency than LSTM (Section 4).
- A timestep-scheduled curriculum that stabilises training across progressive difficulty stages, with a quantified stability trade-off characterised against direct final-stage exposure (Section 3, 4.2).
- A quantitative sustainability analysis showing a 6.73% reduction in learned-policy control effort proxy relative to the best recurrent baseline, with training carbon amortised after approximately 475 deployments (Section 5).

The remainder of this report is organised as follows. Section 2 reviews related work; Section 3 defines the POMDP formulation, simulator, policy architecture, and evaluation setup; Section 4 presents results, ablations, and limitations; Section 5 analyses sustainability; and Section 6 concludes the report.

2 Literature Review

2.1 Classical Navigation Methods

Classical USV navigation and collision avoidance methods are fundamentally grounded in geometric and reactive planning. Algorithms such as Artificial Poten-

tial Fields (APF) (15), the Dynamic Window Approach (DWA) (16), and Velocity Obstacles (VO) (17; 18) provide highly efficient and interpretable local planning. Because of their deterministic nature, these approaches have been widely adopted to ensure compliance with strict maritime rules, such as the International Regulations for Preventing Collisions at Sea (COLREGs). Furthermore, advanced extensions of classical search algorithms, such as constrained Fast Marching (FM) methods, have been successfully deployed to model the dynamic behaviour of moving ships and generate reliable trajectories in time-variant maritime environments (19). These planners are used here as interpretable reference baselines, but their reactive geometric assumptions make them less suited to flow-coupled partial observability, where obstacle motion and drift depend on unobserved temporal dynamics.

2.2 Deep Reinforcement Learning in Marine Navigation

Deep reinforcement learning (DRL) has been widely explored for end-to-end control from sensor observations. Continuous-control algorithms such as DDPG (20) and SAC (21) have been applied to USV path planning and station-keeping in the presence of currents and other vessels (22; 3). To handle complex obstacle geometries, other works have integrated Convolutional Neural Networks (CNNs) with DRL, converting LiDAR or radar returns into 2D grid maps or collision threat layers (7). While treating collision avoidance as a visual recognition task improves spatial awareness, CNN-based encoders can significantly increase computational overhead and inference latency on embedded hardware. More recently, distributional RL has been used to explicitly model return uncertainty by employing Implicit Quantile Networks (IQN) for USV navigation under perturbations, demonstrating improved robustness to stochastic disturbances (9; 10). By approximating the full return distribution, rather than only its expectation, these methods enable risk-aware and safer decision making. However, they do not by themselves provide temporal state estimation under partial observability. Nevertheless, most of these methods are evaluated in environments with static or weakly dynamic obstacles and do not explicitly target long-horizon temporal reasoning in time-varying flow fields. Thus, while CNN and distributional RL methods improve spatial or uncertainty representation, they do not directly address the recurrent belief estimation required to infer latent drift and dynamic obstacle motion from observation history (2).

2.3 Temporal Architectures for Navigation

Partially observable navigation tasks require policies capable of integrating information over time. Early DRL

approaches addressed this by appending recurrent layers such as Long Short-term Memory (LSTM) (14) or Gated Recurrent Units (GRUs) (23) to actor-critic architectures. This yields recurrent PPO or SAC variants that maintain a latent belief over hidden state. While these baselines improve performance over feed-forward policies in standard POMDP settings, their fixed discrete-time updates may be less naturally matched to systems with multi-scale continuous-time dynamics.

To capture long-range dependencies in multi-agent and dynamic scenes, more recent work has introduced transformer-based policies with spatio-temporal attention. MarineFormer (8) is a prominent example for USV navigation, using a combination of spatial graph attention and temporal self-attention to model interactions between the vessel, obstacles, and flow. While such models achieve strong performance in simulation, their quadratic attention complexity and large parameter counts raise concerns for real-time deployment on embedded marine hardware.

Closed-form Continuous-time (CfC) networks (11; 24) have been shown to provide robust sequence modelling and out-of-distribution generalisation for robotic control tasks, including autonomous drone navigation from raw visual input at a fraction of the computational cost of deep recurrent or transformer-based models (25). This motivates evaluating CfC as a lightweight temporal model for USV navigation.

2.4 Safety, Robustness, and Curriculum Learning

Safety is not treated as a formal guarantee; instead, collision avoidance is encouraged through reward design, curriculum exposure, and empirical evaluation of collision-related metrics. Within the POMDP formulation, the reward function defines the behavioural incentives available to the agent during policy optimisation. Therefore, it must balance goal-directed progress with safety-relevant penalties, such as collisions, boundary violations, and operation near obstacles. Reward shaping is commonly used in reinforcement learning to provide denser learning signals, although poorly designed shaping terms can bias the learned behaviour away from the intended task objective (26).

To mitigate these issues, curriculum learning is increasingly used as a practical robustness mechanism (13). By progressively increasing environment difficulty, curriculum training can reduce early failure-dominated exploration. However, existing literature lacks a framework that combines lightweight, continuous-time memory (CfC) with progressive curriculum training to navigate flow-driven environments without relying on external heuristic safety filters.

3 Problem Formulation and Experimental Setup

This section formulates the navigation task as a POMDP and defines the simulated environment, vehicle dynamics, sensing model, observation/action spaces, reward function, and termination conditions.

3.1 Problem Formulation

3.1.1 POMDP

The USV navigation task is formulated as a POMDP in a controlled vortex-flow simulation environment.

$$\mathcal{M} = \langle S, A, O, T, Z, R, \gamma, b_0 \rangle \quad (1)$$

where S is the latent simulator state space, A the discrete action space, O the observation space, $T(s_{t+1} | s_t, a_t)$ the transition model, $Z(o_t | s_t, a_{t-1})$ the observation model, R the reward function, $\gamma \in (0, 1)$ the discount factor, and b_0 the initial belief over latent states.

Because s_t is not directly observed, the decision variable is the belief state $b_t(s) = P(s_t = s | o_{1:t}, a_{0:t-1})$, which summarises the action and observation history. In a model-based POMDP, this belief would be updated by:

$$b_t(s') = \eta Z(o_t | s', a_{t-1}) \sum_{s \in S} T(s' | s, a_{t-1}) b_{t-1}(s)$$

where η is a normalising constant. In this work, the transition and observation models are not solved analytically; instead, the recurrent policy learns a compressed belief representation from the observation history.

At time t , the latent state $s_t \in \mathcal{S}$ comprises the USV pose and velocity, goal location, the positions and velocities of all obstacles, and the parameters of the surrounding vortex cores. The policy does not observe s_t directly; instead, it receives a local robot-frame observation o_t . The agent observes $o_t \in \mathcal{O}$, constructed from on-board sensors as detailed in Section 3.1.3, and selects an action $a_t \in \mathcal{A}$. The environment evolves according to

$$s_{t+1} \sim P(\cdot | s_t, a_t),$$

and the agent receives a scalar reward $r_t = R(s_t, a_t)$. The objective is to maximise the expected discounted return

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t r_t \right].$$

over episodes of length at most T , where T is the maximum episode length and episodes may terminate earlier due to goal reaching, collision, or out-of-boundary.

3.1.2 Environment and USV Configuration

Flow Model: The simulated marine environment generates dynamic current flows modelled as Rankine vortices. For a point located at radial distance r from vortex core i , with circulation strength Γ_i and core radius R_i , the induced tangential flow velocity $u_{\theta,i}(r)$ is defined as:

$$u_{\theta,i}(r) = \frac{\Gamma_i}{2\pi R_i} \times \begin{cases} \frac{r}{R_i} & \text{if } r \leq R_i \\ \frac{R_i}{r} & \text{if } r > R_i \end{cases}$$

The local water velocity $v_c(x, y) = (v_{c,x}, v_{c,y})$ experienced by the USV is the vector superposition of the velocities induced by all active vortices in the environment.

Furthermore, the flow model captures topological fluid changes via vortex merging. When two co-rotating vortices approach each other such that their separation distance $d_{i,j}$ falls below a critical spatial threshold $\lambda(R_i + R_j)$, they amalgamate into a single new vortex k . To retain basic vortex-consistency while keeping the simulator simple, the merged vortex conserves total circulation and the centre of vorticity:

$$\Gamma_k = \Gamma_i + \Gamma_j \quad (2)$$

$$\mathbf{p}_k(t) = \frac{\Gamma_i \mathbf{p}_i(t) + \Gamma_j \mathbf{p}_j(t)}{\Gamma_i + \Gamma_j} \quad (3)$$

$$R_k = R_i = R_j \quad (4)$$

These rules conserve total circulation and the centre of vorticity but hold the core radius fixed. This simplification keeps the simulator computationally light, since the goal is controlled flow variation rather than faithful reproduction of merging hydrodynamics.

USV dynamics: The USV is modelled as a planar rigid body with state (x, y, θ, v) , where $(x, y) \in \mathbb{R}^2$ denotes the position in the world frame, $\theta \in [0, 2\pi)$ is the heading angle, and $v \geq 0$ is the forward speed. Control is applied at discrete intervals of duration $\Delta t = N dt$, where dt is the simulator integration step and N is the number of low-level integration steps executed per high-level action.

Each discrete action index $a_t \in \{0, \dots, 8\}$ maps to a pair (a, ω) of commanded linear acceleration and yaw rate drawn from the finite sets $\mathcal{A}_v = \{-0.4, 0, 0.4\} \text{ m/s}^2$ and $\mathcal{A}_\omega = \{-\pi/6, 0, \pi/6\} \text{ rad/s}$, so that $|\mathcal{A}| = |\mathcal{A}_v| |\mathcal{A}_\omega|$. A discrete action set is used to reduce exploration complexity and enable stable comparison across recurrent PPO agents, while still capturing the main

surge and yaw manoeuvres required for local navigation.

Between action decisions, the simulator integration step is $dt = 0.1 \text{ s}$ with $N = 10$ sub-steps per action, giving a control period of $\Delta t = 1.0 \text{ s}$. At each low-level step of duration dt , the forward speed v obeys a first-order model with linear drag,

$$v_{k+1} = \text{clip}(v_k + (a - kv_k) dt, 0, v_{\max}),$$

where $k = \max(\mathcal{A}_v)/v_{\max} = 0.2 \text{ s}^{-1}$ is a constant drag coefficient computed from the maximum acceleration ($\max(\mathcal{A}_v) = 0.4 \text{ m/s}^2$), and $v_{\max} = 2 \text{ m/s}$ is the speed limit. The commanded yaw rate ω is integrated directly,

$$\theta_{k+1} = \theta_k + \omega dt$$

with θ wrapped back into $[0, 2\pi)$ after each update.

Let $v_c(x, y) = (v_{c,x}, v_{c,y})$ denote the current at the USV's position, and let $(\cos \theta, \sin \theta)$ be the unit vector along the heading. At each low-level step the absolute velocity with respect to the seafloor is:

$$\dot{x} = v \cos \theta + v_{c,x}(x, y) \quad \dot{y} = v \sin \theta + v_{c,y}(x, y)$$

which is integrated in Euler form as:

$$x_{k+1} = x_k + \dot{x} dt \quad y_{k+1} = y_k + \dot{y} dt$$

Sensor model: For perception, the USV is equipped with a modelled 360° LiDAR sensor and $N_b = 73$ evenly spaced beams (5° per angle). The beams span an angular sector of width 2π , with beam i oriented at relative angle:

$$\phi_i = -\pi + i \Delta \phi \quad \Delta \phi = \frac{2\pi}{N_b - 1}$$

At each step, the environment computes the absolute beam directions $\theta + \phi_i$ and casts each beam from the robot's position as a ray of maximum range $R = 10 \text{ m}$.

For each beam, the simulator computes the nearest line-circle intersection with any circular obstacle within range. Hit beams return the robot-frame reflection point, while no-hit beams are encoded as zero vectors in the learned observation. The resulting LiDAR component is therefore $S_t \in \mathbb{R}^{2 \times N_b}$.

3.1.3 Observation and Action Spaces

The agent does not observe the full environment state s_t . Instead, it receives an observation vector $o_t \in \mathbb{R}^{d_o}$ constructed from on-board sensors and localised spatial summaries. The core components are: the absolute USV velocity in the robot frame $v_t^{\text{abs}} \in \mathbb{R}^2$, the relative goal position $g_t \in \mathbb{R}^2$, the local water current velocity

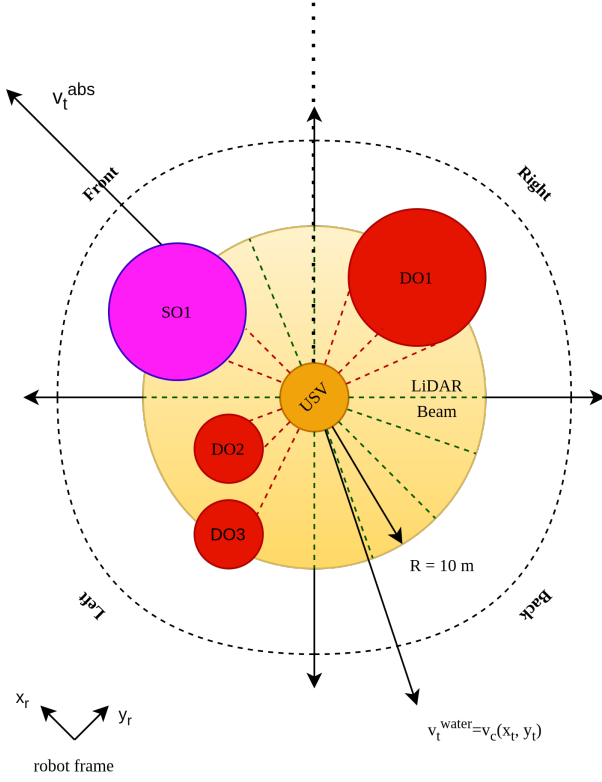


Figure 1: Robot-frame sensing model. The USV uses a 360° LiDAR sensor (yellow) with $N_b = 73$ beams and range $R = 10$ m. Static obstacles (SO) and dynamic obstacles (DO) are circular; red dashed beams denote obstacle intersections forming S_t , while green dashed beams indicate unobstructed ray directions. Sector labels define the front/back/left/right summaries used for d_t^{dir} and c_t .

$v_t^{\text{water}} \in \mathbb{R}^2$, and a flattened LiDAR pointcloud of per beam reflection points $S_t \in \mathbb{R}^{2 \times N_b}$.

To mitigate the inherent sparsity of raw sensor returns, the observation is augmented with two engineered spatial summaries, both strictly bounded by the maximum sensor range R to preserve realistic sensing constraints. The first is a directional obstacle summary $d_t^{\text{dir}} \in \mathbb{R}^{12}$, comprising four 2D sector signals and four sector-validity indicators for the front, back, left, and right regions in the robot frame (Figure 1). Obstacles lying on quadrant boundaries are represented in adjacent buckets for spatial continuity, and each bucket encodes the mean relative velocity of dynamic agents, falling back to the direction of the nearest static obstacle in their absence. The second is a side-wise clearance vector $c_t \in \mathbb{R}^4$, encoding the minimum boundary-to-boundary distance between the USV hull and any obstacle in the front, back, left, and right sectors. For each sector, the clearance is computed as:

$$\text{clearance} = \|p_{\text{obs}} - p_{\text{usv}}\|_2 - r_{\text{obs}} - r_{\text{usv}},$$

clamped to be non-negative and normalised by R .

Thus, the full observation is:

$$o_t = [v_t^{\text{abs}}, g_t, v_t^{\text{water}}, S_t, d_t^{\text{dir}}, c_t]^\top \in \mathbb{R}^{d_o},$$

where the dimension of the observation space is:

$$d_o = 2 + 2 + 2 + 2N_b + 12 + 4 = 168.$$

3.1.4 Reward and Termination

Each episode starts from a random start pose and a random goal location satisfying a minimum start-goal distance. The reward at time t is:

$$r_t = r_{\text{time}} + r_{\text{progress}} + r_{\text{align}} + r_{\text{clearance}} + r_{\text{stall}} + r_{\text{terminal}}$$

Where the role of each component is defined in Table 1 below.

Table 1: Reward components and empirical hyperparameter weights used during PPO training.

Term	Role	Definition	Value
r_{time}	Step penalty	r_{step}	-0.05
r_{progress}	Progress	$w_p(d_{t-1} - d_t)$	$w_p = 2.0$
r_{align}	Alignment	$w_a \hat{g}_t^\top \hat{v}_t$	$w_a = 0.05$
$r_{\text{clearance}}$	Proximity	$-w_c(c_{\text{safe}} - c_{\text{min},t})$ if $c_{\text{min},t} < c_{\text{safe}}$	$w_c = 0.2$ $c_{\text{safe}} = 2.0$
r_{stall}	Stagnation	r_{stall} if progress < ϵ_{stall}	$r_{\text{stall}} = -0.02$ $\epsilon_{\text{stall}} = 0.01$
r_{terminal}	Terminal	$+r_{\text{goal}}$ or $r_{\text{collision}}$	$+100.0$ or -50.0

For the reward term $r_{\text{clearance}}$, $c_{\text{min},t}$ denotes the nearest boundary-to-boundary obstacle clearance around the USV. Episodes terminate upon reaching the goal, collision with an obstacle, leaving the boundaries, or exceeding a maximum step limit. In all termination cases except reaching the goal, the agent receives the same terminal penalty of -50.

3.2 Methodology

The proposed framework addresses maritime partial observability using a hierarchical spatio-temporal architecture. The observation vector is separated into specialised latent embeddings so that high-dimensional LiDAR inputs do not dominate lower-dimensional kinematic, goal, and current features before recurrent belief updating.

3.2.1 Multi-Branch Feature Extraction

The observation o_t is decomposed into three functional branches: kinematic, perception, and dynamic-awareness. Each branch uses an MLP bottleneck with

SiLU activation and Layer Normalisation, sharing the same transformation form but using branch-specific parameters:

$$\Phi_b(\mathbf{x}) = \text{SiLU}(\text{LN}(\mathbf{W}_2 \cdot \text{SiLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2))$$

Where $\text{SiLU}(x) = x \cdot \sigma(x)$. The specific feature extraction process is defined as follows:

Kinematic Branch: This branch processes low-dimensional robot-frame navigation quantities, including the USV absolute velocity $\mathbf{v}_t^{\text{abs}}$, the relative goal vector $\mathbf{g}_t$, and the local water current $\mathbf{v}_t^{\text{water}}$. These inputs are projected into a navigation embedding $\mathbf{h}_{\text{nav}}$:

$$\mathbf{h}_{\text{nav}} = \Phi_{\text{nav}}([\mathbf{v}_t^{\text{abs}}, \mathbf{g}_t, \mathbf{v}_t^{\text{water}}]^\top) \quad (5)$$

Perception Branch: This branch maps the high-dimensional LiDAR reflection vector S_t to capture the global geometric structure of the surrounding area within the sensor range. The mapping extracts a perceptual latent representation $\mathbf{h}_{\text{perc}}$:

$$\mathbf{h}_{\text{perc}} = \Phi_{\text{perc}}(S_t) \quad (6)$$

Dynamic Awareness Branch: Acting as a local risk-awareness branch, this stream processes the engineered 16D sector summary. This consists of four 2D sector signals for front/back/left/right regions, four sector-validity indicators, and four normalised clearance values. These components are represented as d_t^{dir} and c_t , and are projected into an awareness embedding $\mathbf{h}_{\text{dyn}}$:

$$\mathbf{h}_{\text{dyn}} = \Phi_{\text{dyn}}([\mathbf{d}_t^{\text{dir}}, \mathbf{c}_t]^\top) \quad (7)$$

The resulting embeddings are concatenated to form the dense, multimodal state representation $\mathbf{z}_t$ from Equation 5, 6 and 7:

$$\mathbf{z}_t = [\mathbf{h}_{\text{nav}} \parallel \mathbf{h}_{\text{perc}} \parallel \mathbf{h}_{\text{dyn}}] \quad (8)$$

where $\parallel$ denotes the concatenation operator. This fused vector $\mathbf{z}_t$ provides a per time step multimodal representation for the subsequent recurrent core.

3.2.2 Recurrent Architecture

Following the POMDP belief formulation, the recurrent hidden state h_t is treated as a learned finite-dimensional approximation of the belief b_t . The encoder maps the instantaneous observation to $z_t = \phi_\theta(o_t)$, and the recurrent core updates

$$\hat{b}_t \equiv h_t = f_\theta(z_t, h_{t-1}),$$

allowing the policy to summarise latent flow effects and obstacle-motion trends from observation history.

Discrete-time (LSTM/GRU): Two widely adopted discrete-time recurrent architectures are used as baselines to compare alternative forms of temporal memory.

- **Long Short-Term Memory (LSTM):** The LSTM (14) maintains a hidden state $\mathbf{h}_t$ and a cell state $\mathbf{c}_t$ via three gating mechanisms (input, forget, and output) to manage long-term dependencies:

$$[h_t, c_t^{\text{LSTM}}] = \text{LSTM}_\theta(z_t, h_{t-1}, c_{t-1}^{\text{LSTM}}),$$

- **Gated Recurrent Unit (GRU):** The GRU (23) simplifies the recurrent structure by merging the cell and hidden states into a single latent vector $\mathbf{h}_t$, utilising reset and update gates to control information flow with fewer parameters than the LSTM:

$$h_t = \text{GRU}_\theta(z_t, h_{t-1}).$$

While these models improve performance in POMDP settings, their fixed discrete-time gating may be less naturally matched to sampled systems with multi-scale continuous-time dynamics.

Continuous-time (CfC): To test whether continuous-time recurrent parameterisation benefits the sampled vortex-flow task, this research evaluates a Closed-form Continuous-time (CfC) neural network (11; 24). Unlike traditional RNNs, the CfC utilises a liquid time-constant mechanism that approximates the solution of continuous-time differential equations in closed form. For CfC, the belief update is parameterised as a closed-form continuous-time transition rather than a purely discrete gate. Let $u_t = [z_t, h_{t-1}]$. The CfC cell computes two candidate hidden transformations and a time-dependent interpolation gate:

$$q_t = \sigma(f_q(u_t, \Delta t)) \quad \tilde{h}_t^A = f_A(u_t) \quad \tilde{h}_t^B = f_B(u_t) \\ h_t = q_t \odot \tilde{h}_t^A + (1 - q_t) \odot \tilde{h}_t^B$$

Here, q_t acts as a learned continuous-time update gate controlled by the encoded observation, previous hidden state, and elapsed time Δt . This gives the recurrent belief update a continuous-time inductive bias, which is expected to be better matched to the smooth vortex-induced drift than fixed discrete-time gating alone. This is not an explicit hydrodynamic model; it is a learned continuous-time latent-dynamics prior used to approximate the POMDP belief update. The complete architecture is summarised in Figure 2.

3.2.3 Policy Architectures

To isolate the effect of temporal modelling, the CfC, LSTM, and GRU agents share the same observation vector, multi-branch encoder, PPO objective, reward function, curriculum, training budget, and actor-critic heads. The recurrent core is the only varied neural component.

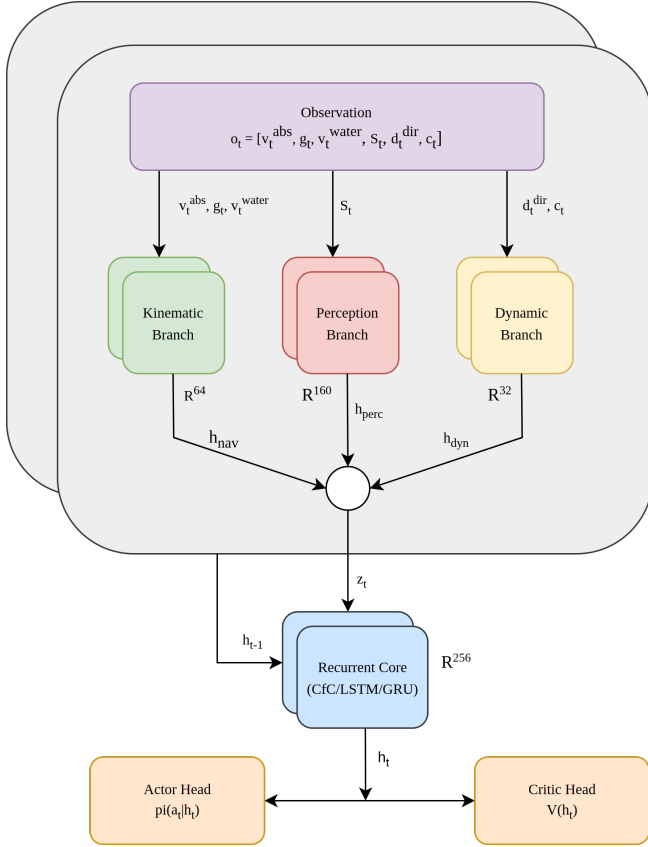


Figure 2: Policy architecture. The observation is split into kinematic, perception, and dynamic-awareness branches, fused into z_t , and passed to a recurrent core (CfC/LSTM/GRU) before the shared actor-critic heads.

3.2.4 Timestep-Scheduled Curriculum Training

All architectures are trained via PPO using Ray RLlib (27; 28) and PyTorch (29) on an RTX 4060 GPU Laptop. To maintain fair architectural comparison, all models share a fixed interaction budget and identical hyperparameters (Table 2). To reduce early failure-dominated exploration, training employs a four-stage, timestep-scheduled curriculum (Figure 3). Rather than relying on online performance thresholds, environment difficulty scales deterministically with training steps by progressively increasing obstacle density, introducing dynamic obstacles, and extending the start-goal separation. The vortex count remains fixed across all stages so that the curriculum primarily varies obstacle density, dynamic-obstacle count, and start-goal separation.

3.3 Experimental Setup

3.3.1 Evaluation Framework

Following 6M training time steps, each frozen policy is evaluated over 500 independent episodes per configuration using unseen random seeds. This strict separation of training and evaluation distributions assesses both in-distribution robustness and out-of-distribution generalisation.

Table 2: Core PPO training settings.

Setting	Value
Algorithm	PPO (RLlib/PyTorch)
Budget	6M env. steps
Training seed / env. offsets	2 / 2–33
Parallel envs	32 (8 runners $\times$ 4 envs)
Batch / minibatch	8192 / 1024
SGD epochs	8
Learning rate	3×10^{-5}
Clip / KL target	0.15 / 0.01
Entropy / grad clip	0.015 / 1.0
VF clip	10.0
Seq. length	64
Varied module	CfC, LSTM, GRU

Table 3: Configuration parameters for the four-stage timestep-scheduled curriculum.

Parameter	Stage 1	Stage 2	Stage 3	Stage 4
Steps	$0 \leq T < 1M$	$1 \leq T < 2M$	$2 \leq T < 3M$	$T \geq 3M$
Total obs.	6	8	8	8
Dynamic obs.	0	0	1	4
Min. dist.	35	35	40	40
Num. cores	8	8	8	8

Policy performance is evaluated across four configurations: Dense Static, Dynamic, out-of-distribution (OOD) Dense, and Sensor Robustness.

- **Static:** all obstacles are static, while the USV remains subject to the vortex-induced current field.
- **Dynamic:** matches the final curriculum stage but uses unseen seeds for obstacle placement, vortex initialisation, start-goal pairs, and robot initial state.
- **OOD Dense:** increases total obstacles from 8 to 10 and dynamic obstacles from 4 to 6, exceeding the final curriculum stage.
- **Sensor Robustness:** keeps the Dynamic configuration but reduces sensor range from 10 m to 5 m at evaluation time, without policy fine-tuning.

Table 4 summarises the evaluation parameters across these four configurations.

Table 4: Experimental configurations for in-distribution and OOD evaluation scenarios.

Parameter	S1 Static	S2 Dynamic	S3 OOD Dense	S4 Sensor Robustness
Iterations	500	500	500	500
Total obs.	8	8	10	8
Dynamic obs.	0	4	6	4
Min. distance	40	40	40	40
Vortex cores	8	8	8	8
Sensor range	10	10	10	5

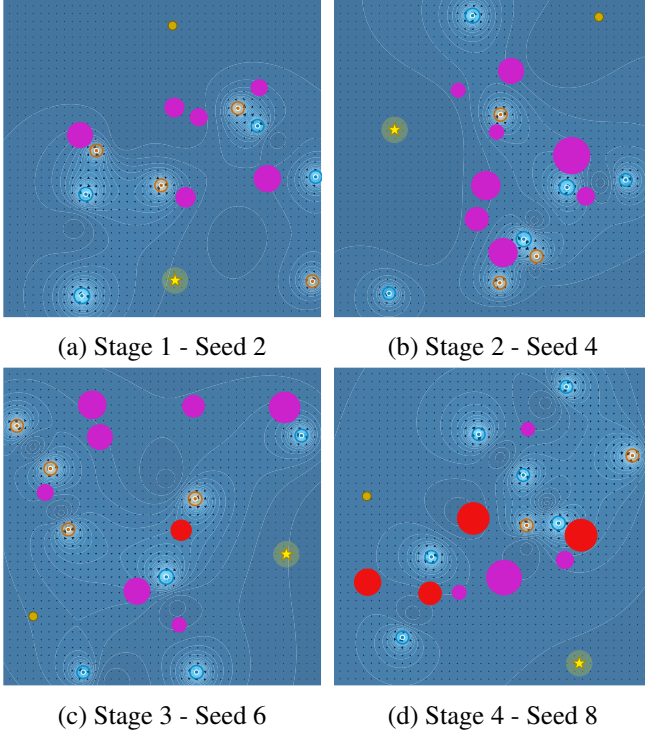


Figure 3: Four-stage curriculum training examples. Yellow markers denote start positions, stars denote goals, purple circles denote static obstacles, red circles denote dynamic obstacles, and cyan/orange cores denote CW/CCW vortices. Difficulty increases through obstacle density, dynamic-obstacle count, and start-goal separation.

3.3.2 Comparative Baselines and Metrics

The proposed policy is compared against recurrent PPO baselines and classical reference planners.

Recurrent Architecture Baseline: The only architectural component varied between agents is the recurrent core, which is implemented as either a GRU, LSTM, or the proposed CfC module. This allows performance differences to be attributed primarily to the temporal representation rather than to differences in training exposure or input processing.

Classical planners: Two classical planners are included as model-based reference baselines: APF and DWA. APF selects actions from goal-attraction and obstacle-repulsion forces, while DWA scores short-horizon candidate actions for goal progress and trajectory safety. Both are adapted to the discrete action set, but use simulator-level obstacle geometry and local current information rather than the learned observation vector.

Metrics are success rate (SR), collision rate (CR), out-of-boundary rate (OOB), mean reward, normalised control effort per episode (E/Ep), and inference latency. SR,

CR, and OOB denote terminal outcomes; E/Ep is a control effort proxy rather than a direct physical energy measurement; latency is average wall-clock time per action on the same hardware.

4 Results and Discussion

The evaluation is organised around three possible sources of performance: temporal modelling, training distribution, and engineered spatial priors. The main evaluation compares CfC+PPO against recurrent and classical baselines, while the ablations isolate the effects of memory, curriculum scheduling, and engineered sector summaries. The results therefore assess not only whether CfC+PPO performs best, but which design assumptions drive the observed robustness.

4.1 Results

4.1.1 Training

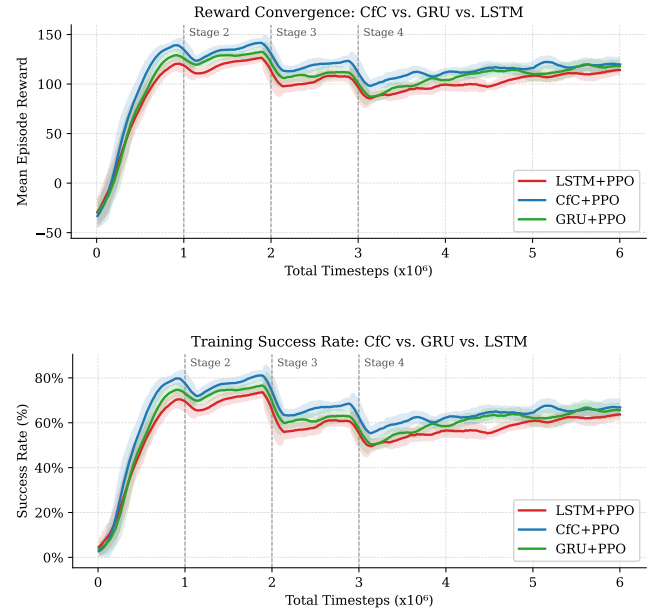


Figure 4: Training performance (mean episodic reward and success rate) across 6M timesteps, highlighting stage-transition dynamics.

After 6M environment time steps, Figure 4 shows temporary reward and success-rate drops at curriculum transitions. These drops are expected because each transition changes the training distribution: Stage 2 increases obstacle density, Stage 3 introduces dynamic obstacles and longer minimum start-goal separation, and Stage 4 increases dynamic traffic. Immediately after each transition, the previously learned policy encounters more collisions, longer paths, and reduced progress reward, producing a short-term reward drop. CfC+PPO learns more slowly in Stage 1, but after later transitions it shows shallower drops and faster recovery, suggesting greater

optimisation stability under flow-coupled distribution shifts. By convergence, the recurrent policies approach similar training plateaus, so the learning curves mainly indicate adaptation behaviour; final navigation reliability is assessed through unseen evaluation episodes.

4.1.2 Evaluation

Table 5: In-distribution navigation metrics across Dense Static and Dynamic environments.

Scenario	Method	SR	CR	OOB	Reward
Static	APF	50.2	46.4	3.4	86.06 ± 101.07
	DWA	60.2	36.4	3.4	105.96 ± 98.68
	GRU+PPO	78.8	11.2	10.0	144.42 ± 81.91
	LSTM+PPO	79.2	14.0	6.8	146.23 ± 79.55
	CfC+PPO	85.4	8.4	6.2	158.31 ± 69.51
Dynamic	APF	49.6	45.8	4.6	85.45 ± 99.72
	DWA	62.4	35.2	2.4	109.53 ± 98.05
	GRU+PPO	70.4	27.6	2.0	129.29 ± 87.36
	LSTM+PPO	69.4	26.6	4.0	127.45 ± 88.72
	CfC+PPO	76.8	20.8	2.4	141.32 ± 81.67

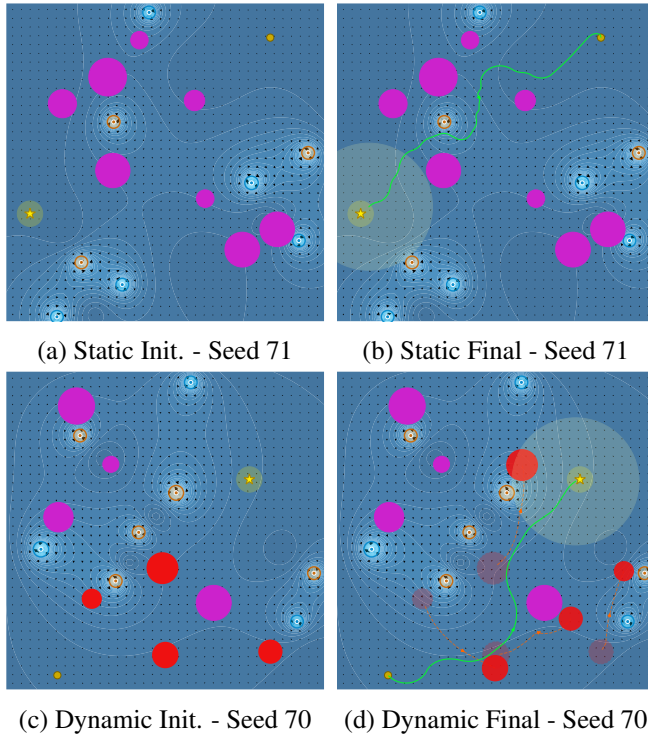


Figure 5: Representative in-distribution CfC trajectories. (a,b) Dense Static evaluation; (c,d) Dynamic evaluation with flow-coupled obstacles.

Table 5 shows that collision, rather than boundary violation, is the dominant failure mode across both in-distribution settings, particularly for APF and DWA. This indicates that the task primarily stresses local risk estimation under flow disturbance rather than simple workspace containment. In Static, CfC+PPO achieves 85.4% SR and 8.4% CR, outperforming LSTM by 6.2% absolute SR and reducing CR by 40% relative to LSTM. This setting is not purely geometric: although obstacles are fixed, vortex-induced drift still accumulates

over time, so the result suggests that temporal state estimation helps correct hidden flow effects even without moving obstacles. In Dynamic, where dynamic obstacles create direct crossing interactions, CfC remains strongest with 76.8% SR and 20.8% CR. LSTM and GRU achieve comparable Dynamic SR (69.4% and 70.4%), suggesting that the choice between these two discrete-time gating mechanisms is not the dominant factor for this task. CfC’s larger margin over both is consistent with its recurrent structure being better matched to the smooth flow-driven dynamics, though confirming this mechanism would require dedicated ablation against time-invariant recurrent variants.

4.1.3 Out-of-Distribution Generalisation

Table 6: Performance under OOD Dense and Sensor Robustness evaluations.

Scenario	Method	SR	CR	OOB	Reward
OOD Dense	APF	40.0	56.0	4.0	65.86 ± 98.40
	DWA	53.8	42.2	4.0	90.84 ± 101.69
	GRU+PPO	59.2	36.4	4.4	106.73 ± 95.42
	LSTM+PPO	60.8	35.6	3.6	108.83 ± 95.53
	CfC+PPO	66.6	29.4	4.0	120.86 ± 91.95
Sensor Robustness	APF	-	-	-	-
	DWA	-	-	-	-
	GRU+PPO	68.0	30.6	1.4	123.88 ± 90.06
	LSTM+PPO	69.6	28.6	1.8	126.75 ± 89.28
	CfC+PPO	75.4	23.2	1.4	138.05 ± 83.83

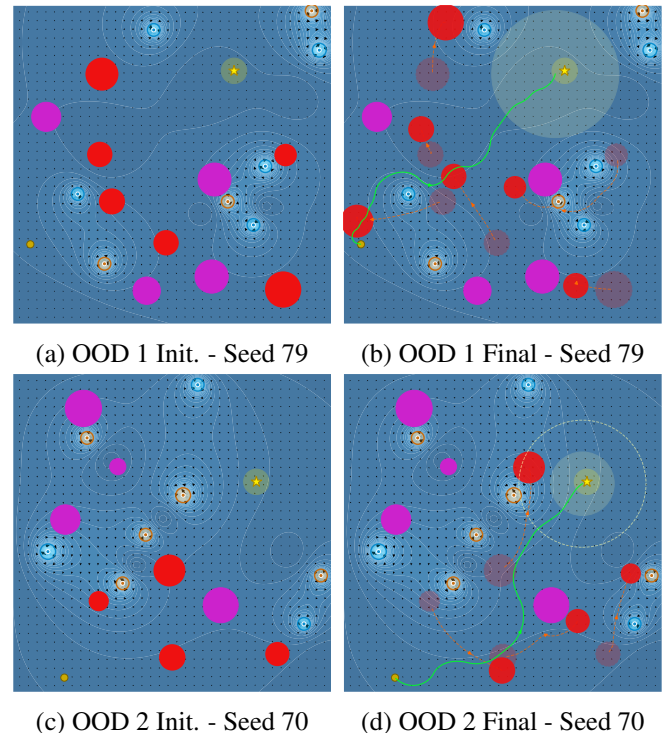


Figure 6: Representative OOD and sensor robustness trajectories. (a,b) OOD Dense illustrates routing under increased clutter. (c,d) Sensor Robustness shows navigation under a restricted perception radius.

Moreover, Table 6 extends this trend to the OOD settings, where CfC+PPO maintains the strongest overall

generalisation performance. In OOD Dense, obstacle count and dynamic-obstacle fraction exceed the curriculum maximum, degrading all learned policies; nevertheless, CfC retains 66.6% SR and 29.4% CR. This pattern is consistent with CfC providing a continuous-time latent-dynamics prior: its hidden state evolves through a learned continuous-time parameterisation, whereas LSTM and GRU rely on fixed discrete-time gated updates. In a simulator governed by smooth vortex-induced drift and flow-coupled obstacle motion, this inductive bias may help preserve latent flow information between observations, contributing to higher SR. APF and DWA degrade more sharply because their reactive decisions do not maintain a temporal estimate of flow-coupled risk. In Sensor Robustness, reducing the sensing range from $R = 10$ m to $R = 5$ m lowers learned-policy performance, but CfC still achieves the highest SR (75.4%) and lowest CR (23.2%). APF and DWA are omitted from the sensor robustness comparison because they access simulator-level obstacle geometry and are therefore not affected by sensor-range degradation in an observation-matched way. Overall, these results suggest that temporal memory partially compensates for reduced instantaneous sensing in simulation, while physical sensor noise and hardware trials remain necessary before deployment-level conclusions can be drawn.

4.2 Ablation Studies and Limitations

4.2.1 Ablation Study 1: The Necessity of Memory

Table 7: Ablation on temporal memory: comparison between the memoryless MLP+PPO baseline and the recurrent CfC+PPO policy under identical training and evaluation settings.

Scenario	Method	SR	CR	Reward	E/Ep	Infer.
Static	MLP+PPO	76.6	13.4	140.71	62.14	1.06
	CfC+PPO	85.4	8.4	158.31	56.45	1.47
Dynamic	MLP+PPO	73.6	22.8	135.22	52.91	1.05
	CfC+PPO	76.8	20.8	141.32	48.92	1.48
OOD Dense	MLP+PPO	59.4	36.6	106.71	48.48	0.99
	CfC+PPO	66.6	29.4	120.86	47.01	1.55

Table 7 shows that recurrence improves success and collision avoidance, but the gain is scenario-dependent. The largest SR improvement occurs in Static (+8.8%), where vortex-induced drift accumulates without strong instantaneous LiDAR cues. In Dynamic, the gap narrows to +3.2% because moving obstacles generate direct range changes that a memoryless policy can partially react to. OOD Dense restores a larger gap (+7.2%), indicating that temporal state becomes more valuable when clutter makes instantaneous observations harder to disambiguate. Thus, recurrent integration is most useful for slow-varying cumulative hazards rather than only for visibly moving obstacles.

4.2.2 Ablation Study 2: The Importance of Curriculum Training

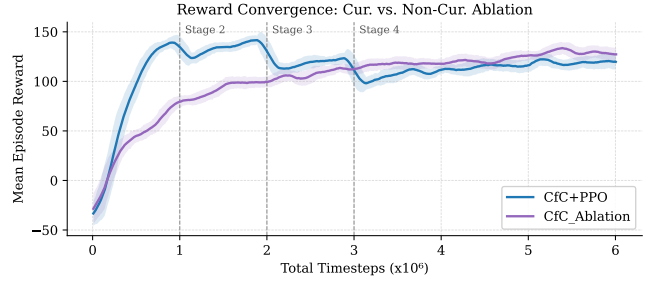


Figure 7: Reward convergence for curriculum and direct final-stage training. The non-curriculum agent learns more slowly initially but reaches a slightly higher late-stage reward.

Figure 7 shows that direct final-stage training learns more slowly at first, consistent with harder early exploration under sparse rewards. The non-curriculum agent later recovers within the 6M-timestep budget and reaches a slightly higher convergence level.

Table 8: Ablation on curriculum schedule: comparison between direct final-stage training (Non-cur) and the four-stage timestep-scheduled curriculum (Cur.) under the same total training budget.

Scenario	Training	SR	CR	Reward	E/Ep	Infer.
Static	Non-cur.	80.6	13.4	148.86	53.60	1.58
	Cur.	85.4	8.4	158.31	56.45	1.47
Dynamic	Non-cur.	76.6	18.4	140.57	49.53	1.68
	Cur.	76.8	20.8	141.32	48.92	1.48
OOD Dense	Non-cur.	72.6	23.0	133.97	49.12	1.49
	Cur.	66.6	29.4	120.86	47.01	1.55

However, Table 8 exposes a critical trade-off rather than simple curriculum superiority. By training exclusively in extreme conditions, the non-curriculum agent develops a highly specialised policy, actually outperforming the curriculum model by 6.0% in the OOD Dense scenario (72.6% vs. 66.6% SR) and demonstrating strong zero-shot transfer (80.6% SR) to the unseen Dense Static setting.

Overall, the four-stage curriculum acts as a regulariser, instantiating the classical plasticity-stability trade-off in RL (30): the non-curriculum agent retains higher plasticity to specialise for extreme-case distributions, while the curriculum agent gains stability across standard operational domains at the cost of peak OOD performance. For safety-critical maritime deployment, where consistent failure avoidance across varied conditions may outweigh peak performance in a single extreme scenario, the curriculum is a reasonable design choice when standard-domain stability is prioritised. A performance-gated curriculum (13) could preserve staged skill acquisition while reducing under-exposure to difficult configurations.

4.2.3 Ablation Study 3: Engineered Spatial Features

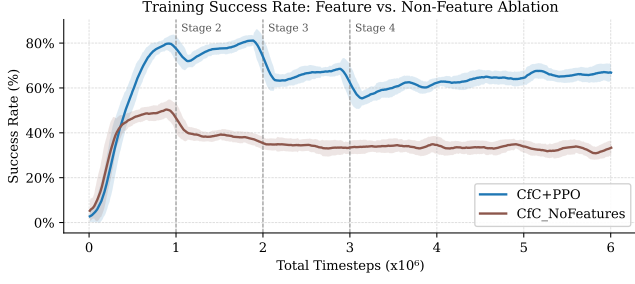


Figure 8: Training success-rate comparison between the full CfC policy and the no-feature CfC ablation trained without engineered spatial features (d_t^{dir} and c_t).

Table 9: Training success-rate comparison between the full CfC policy and the no-feature CfC ablation trained without d_t^{dir} and c_t .

Scenario	Training	SR	CR	Reward	E/Ep	Infer.
Static	w/o d_t^{dir}, c_t	36.8	62.4	60.11	36.10	1.48
	Full CfC	85.4	8.4	158.31	56.45	1.47
Dynamic	w/o d_t^{dir}, c_t	41.0	58.2	69.33	37.73	1.46
	Full CfC	76.8	20.8	141.32	48.92	1.48
OOD Dense	w/o d_t^{dir}, c_t	30.2	68.4	47.07	34.11	1.54
	Full CfC	66.6	29.4	120.86	47.01	1.55

Removing d_t^{dir} and c_t causes a large SR drop and CR increase across all scenarios, indicating that the engineered spatial summaries provide a strong local geometric prior. This suggests that the current MLP-based perception branch does not extract sufficient local geometric information from the raw LiDAR reflection vector alone for reliable recurrent decision-making. This does not invalidate the architecture; it clarifies the division of labour within it: CfC provides temporal integration, while d_t^{dir} and c_t make obstacle direction and clearance explicitly available to the recurrent core. This motivates stronger LiDAR encoders that can learn these spatial risks directly and reduce reliance on manually engineered signals.

4.2.4 Discussion and Limitations

The ablations clarify that the proposed agent is a structured hybrid policy rather than a purely end-to-end LiDAR controller. Temporal recurrence improves robustness to cumulative hidden drift, curriculum training regularises standard-domain behaviour, and the engineered summaries d_t^{dir} and c_t provide the local geometric prior required for reliable collision avoidance. The feature ablation results indicate that the current MLP perception branch does not extract sufficient obstacle direction and clearance information from the raw LiDAR reflection vector alone, motivating future PointNet (31) or graph-based encoders (32).

Several validation limitations remain. The timestep-scheduled curriculum advances by elapsed training steps rather than demonstrated competence, which can under-expose the policy to the hardest configurations; performance-gated progression could reduce this issue (13). The reward shaping terms r_{align} and $r_{\text{clearance}}$ encourage collision avoidance but provide no formal safety guarantee and do not encode COLREGs. The 8.4% residual collision rate in CfC’s best scenario (42 of 500 episodes) highlights the gap between simulation performance and the reliability expected for COLREG-compliant operation. The Static SR gap between CfC and LSTM is statistically significant at evaluation level ($z = 2.57$, $p \approx 0.010$), but each architecture is represented by a single trained policy instance using training seed 2 and 32 parallel environments with deterministic worker/vector seed offsets, so this does not capture training-run variance.

The reported results should therefore be interpreted as strong simulation-based evidence rather than definitive proof of architectural superiority. Establishing deployment-level validity would require independent training seeds, confidence intervals across policy initialisations, physical sensor-noise modelling, hardware-in-the-loop testing, and ultimately real-world USV trials.

5 Sustainability Analysis

Energy consumption is quantified via a normalised control effort proxy accumulated over each episode:

$$E/Ep = \sum_{t=1}^T \left[\left(\frac{|a_t|}{a_{\text{max}}} \right)^2 + \left(\frac{|\omega_t|}{\omega_{\text{max}}} \right)^2 \right] \quad (9)$$

where a_t and ω_t are the commanded surge acceleration and yaw rate at decision step t , and T is the episode length. The quadratic scaling reflects non-linear actuator power consumption with effort. Cross-agent comparisons are valid as all policies share an identical action space and control period $\Delta t = 1.0$ s.

Table 10: Operational (E/Ep) and computational (Infer.) sustainability metrics across all scenarios.

Scenario	Metric	APF	DWA	GRU	LSTM	CfC
Dense Static	E/Ep	27.38	36.69	62.26	58.36	56.45
	Infer.	0.42	15.57	1.94	2.78	1.47
Dynamic	E/Ep	27.79	36.75	52.45	51.16	48.92
	Infer.	0.43	17.56	1.86	2.69	1.49
OOD Dense	E/Ep	25.69	34.25	48.66	47.67	47.01
	Infer.	0.38	17.76	1.90	2.79	1.55
Sensor Robustness	E/Ep	–	–	50.07	49.47	45.24
	Infer.	–	–	2.18	2.59	1.49

The CfC+PPO policy delivers favourable computational sustainability. Its 45% lower inference latency relative

to LSTM, while well within the 1-second control budget on the desktop GPU used here, suggests headroom for deployment on power-constrained edge hardware; direct processor power measurements would be required to quantify the corresponding energy benefit. Beyond computational gains, smoother trajectory planning can reduce the control effort proxy in simulation relative to reactive stop-and-go manoeuvring. From Table 10, CfC+PPO achieves the lowest learned-policy control effort proxy while also maintaining the highest SR. This suggests that its trajectories require less corrective actuation in simulation, although the proxy does not directly measure propulsion energy and the mechanism was not isolated experimentally. However, a critical and often overlooked cost is that of training itself: training the three learned policy variants with 3 ablation studies required approximately 4.5 hours of active GPU computation per session on an RTX 4060 Laptop (TDP: 115 W), totalling an estimated $0.115 \times 6 \times 4.5 = 3.105$ kWh. At the UK grid carbon intensity of approximately 200 gCO₂/kWh, this corresponds to roughly 0.621 kgCO₂, a modest but non-negligible overhead that scales significantly if iterative hyperparameter searches or retraining for new deployment environments are required.

To quantify further, the following is grounded in the Clearpath Heron USV, a representative research-class autonomous surface vessel with a battery capacity of 417.6 Wh (14.4 V $\times$ 29 Ah) and a nominal power draw of $P = 167$ W at cruise speed (33). In the Dynamic scenario, CfC+PPO achieves 48.92 energy units per episode against GRU's 52.45, a 6.73% reduction ($\Delta E_{\text{proxy}} = 3.53$ units/ep). Over a representative mission of $t = 350$ s, the platform expends:

$$E_{\text{mission}} = Pt = 167 \times 350 = 58,450 \text{ J} = 16.2 \text{ Wh}$$

Applying the 6.73% reduction yields $\Delta E_{\text{mission}} \approx 1.09$ Wh saved per mission. Over 100 repeated deployments:

$$\Delta E_{100} = 100 \times 1.09 \text{ Wh} = 109 \text{ Wh}$$

Over 100 deployments, this accumulated saving is approximately 109 Wh, equivalent to 26.1% of one 417.6 Wh battery charge. This suggests a modest but meaningful operational gain on a fixed energy budget, subject to hardware validation. Using the single-run CfC training footprint of approximately 0.10 kg CO₂, the corrected 6.73% propulsion saving, equivalent to approximately 1.09 Wh per 350 s mission, requires approximately 475 repeated deployments to offset the energy cost of one CfC training run. If the full six-session experimental training cost is included, the break-even point increases to approximately 2850 deployments. Therefore, the ecological justification for learned policies depends on amortisation across sustained operational programmes rather than isolated deployments.

6 Conclusions

This research developed a lightweight recurrent RL policy for USV navigation in a controlled vortex-flow simulator. The CfC+PPO policy outperformed GRU and LSTM baselines across the evaluated scenarios, reducing collision rates by up to 40% in Static evaluation and lowering the learned-policy control effort proxy by 6.73% in Dynamic flow. Ablations show that the performance gain arises from a combination of recurrent temporal belief estimation and engineered spatial summaries, rather than from CfC recurrence alone. The framework therefore provides promising simulation-based evidence for adaptive USV navigation under partial observability, while physical validation, COLREGs-aware constraints, multi-seed training, and stronger LiDAR encoders remain necessary before deployment-level claims can be made.

Future work should integrate stronger perception backbones, such as Graph Neural Networks, to extract geometric context directly from raw LiDAR arrays and reduce reliance on engineered spatial summaries. The framework should also be transferred to higher-fidelity ROS2-based simulation (34) in Gazebo (35; 36) or MuJoCo (37), enabling 6-DOF testing under unmodelled hydrodynamic disturbances, realistic sensor noise, and hardware-in-the-loop validation.

7 Declarations

7.1 Acknowledgements

Special thanks go to my supervisor Dr. Yuanchang Liu for his supervision and support. His maritime expertise and feedback were vital to the completion of this work. I am truly appreciative of the time he invested in reviewing my drafts and for the academic freedom he allowed me to explore this research area.

7.2 Declaration

I, Minh Trinh, confirm the use of generative AI (Gemini 3.1 by Google) in an assistive capacity (Category 2) for English proofreading.

References

- [1] Lou J, Xu M, Yue L, Feng W, Wang H. Advances and frontiers of key technologies in intelligent navigation for unmanned surface vehicles. Chinese Journal of Ship Research. 2025;20(1):3-14. Available from: <https://www.sciopen.com/article/10.19693/j.issn.1673-3185.04348>.

- [2] Gadhvi R, Manjanna S. Khalasi: Energy-Efficient Navigation for Surface Vehicles in Vortical Flow Fields; 2025. Available from: <https://arxiv.org/abs/2512.06912>.
- [3] Cheng Y, Zhang W. Concise deep reinforcement learning obstacle avoidance for underactuated unmanned marine vessels. *Neurocomputing*. 2018;275:219-33.
- [4] Woo J, Yu C, Kim N. Deep reinforcement learning-based controller for path following of an unmanned surface vehicle. *Ocean Engineering*. 2019;183:155-66. Available from: <https://www.sciencedirect.com/science/article/pii/S0029801819302203>.
- [5] Wu X, Wei C, Guan D, Ji Z. Risk-aware deep reinforcement learning for mapless navigation of unmanned surface vehicles in uncertain and congested environments. *Ocean Engineering*. 2025;322:120446. Available from: <https://www.sciencedirect.com/science/article/pii/S0029801825001611>.
- [6] Sun X, Li G, Liu Z, Zhang L, Yu H, Song D, et al. Path planning algorithm for unmanned surface vessels based on COLREGs and Meta-TD3. *Ocean Engineering*. 2025;334:121580. Available from: <https://www.sciencedirect.com/science/article/pii/S0029801825012867>.
- [7] Sun P, Yang C, Zhou X, Wang W. Path Planning for Unmanned Surface Vehicles with Strong Generalization Ability Based on Improved Proximal Policy Optimization. *Sensors*. 2023;23(21). Available from: <https://www.mdpi.com/1424-8220/23/21/8864>.
- [8] Kazemi E, Gao D, Soltani I. MarineFormer: A Spatio-Temporal Attention Model for USV Navigation in Dynamic Marine Environments. *arXiv preprint arXiv:241013973*. 2024. Available from: <https://arxiv.org/abs/2410.13973>.
- [9] Zhang Z, Yang M, Xie S, Xiong H. Perturbation-mitigated USV Navigation with Distributionally Robust Reinforcement Learning; 2025. Available from: <https://arxiv.org/abs/2512.00030>.
- [10] Lin X, McConnell J, Englot B. Robust Unmanned Surface Vehicle Navigation with Distributional Reinforcement Learning. In: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS); 2023. p. 6185-91.
- [11] Hasani R, Lechner M, Amini A, Rus D, Grosu R. Liquid Time-constant Networks; 2020. Available from: <https://arxiv.org/abs/2006.04439>.
- [12] Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal Policy Optimization Algorithms; 2017. Available from: <https://arxiv.org/abs/1707.06347>.
- [13] Narvekar S, Peng B, Leonetti M, Sinapov J, Taylor ME, Stone P. Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey; 2020. Available from: <https://arxiv.org/abs/2003.04960>.
- [14] Hochreiter S, Schmidhuber J. Long Short-Term Memory. *Neural Comput*. 1997 Nov;9(8):1735–1780. Available from: <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [15] Khatib O. Real-time obstacle avoidance for manipulators and mobile robots. In: *Proceedings. 1985 IEEE International Conference on Robotics and Automation*. vol. 2; 1985. p. 500-5.
- [16] Fox D, Burgard W, Thrun S. The dynamic window approach to collision avoidance. *IEEE Robotics Automation Magazine*. 1997;4(1):23-33.
- [17] Fiorini P, Shiller Z. Motion planning in dynamic environments using velocity obstacles. *The international journal of robotics research*. 1998;17(7):760-72.
- [18] Benjamin MR, Curcio JA, Leonard JJ, Newman PM. Navigation of unmanned marine vehicles in accordance with the rules of the road. In: *Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006*; 2006. p. 3581-7.
- [19] Liu Y, Bucknall R, Zhang X. The fast marching method based intelligent navigation of an unmanned surface vehicle. *Ocean Engineering*. 2017;142:363-76. Available from: <https://www.sciencedirect.com/science/article/pii/S0029801817303979>.
- [20] Lillicrap TP, Hunt JJ, Pritzel A, Heess N, Erez T, Tassa Y, et al.. Continuous control with deep reinforcement learning; 2019. Available from: <https://arxiv.org/abs/1509.02971>.
- [21] Haarnoja T, Zhou A, Abbeel P, Levine S. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor; 2018. Available from: <https://arxiv.org/abs/1801.01290>.

- [22] Zhou X, Wu P, Zhang H, Guo W, Liu Y. Learn to Navigate: Cooperative Path Planning for Unmanned Surface Vehicles Using Deep Reinforcement Learning. *IEEE Access*. 2019;7:165262-78. Available from: <https://doi.org/10.1145/1553374.1553380>.
- [23] Chung J, Gulcehre C, Cho K, Bengio Y. Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling; 2014. Available from: <https://arxiv.org/abs/1412.3555>.
- [24] Hasani R, Lechner M, Amini A, Liebenwein L, Ray A, Tschaikowski M, et al. Closed-form continuous-time neural networks. *Nature Machine Intelligence*. 2022 Nov;4(11):992-1003. Available from: <http://dx.doi.org/10.1038/s42256-022-00556-7>.
- [25] Chahine M, Hasani R, Kao P, Ray A, Shubert R, Lechner M, et al. Robust flight navigation out of distribution with liquid neural networks. *Science Robotics*. 2023;8(77):eadc8892. Available from: <https://www.science.org/doi/abs/10.1126/scirobotics.adc8892>.
- [26] Ng AY, Harada D, Russell SJ. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In: *Proceedings of the Sixteenth International Conference on Machine Learning. ICML '99*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc.; 1999. p. 278-287.
- [27] Liang E, Liaw R, Nishihara R, Moritz P, Fox R, Goldberg K, et al. RLlib: Abstractions for Distributed Reinforcement Learning. In: *International Conference on Machine Learning (ICML)*; 2018. Available from: <https://arxiv.org/pdf/1712.09381>.
- [28] Wu Z, Liang E, Luo M, Mika S, Gonzalez JE, Stoica I. RLlib Flow: Distributed Reinforcement Learning is a Dataflow Problem. In: *Conference on Neural Information Processing Systems (NeurIPS)*; 2021. Available from: <https://proceedings.neurips.cc/paper/2021/file/2bce32ed409f5ebcee2a7b417ad9beed-Paper.pdf>.
- [29] Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library; 2019. Available from: <https://arxiv.org/abs/1912.01703>.
- [30] Bengio Y, Louradour J, Collobert R, Weston J. Curriculum learning. In: *Proceedings of the 26th Annual International Conference on Machine Learning. ICML '09*. New York, NY, USA: Association for Computing Machinery; 2009. p. 41-48.
- [31] Qi CR, Su H, Mo K, Guibas LJ. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation; 2017. Available from: <https://arxiv.org/abs/1612.00593>.
- [32] Zhou J, Cui G, Hu S, Zhang Z, Yang C, Liu Z, et al. Graph Neural Networks: A Review of Methods and Applications; 2021. Available from: <https://arxiv.org/abs/1812.08434>.
- [33] Clearpath Robotics. Heron USV - Portable and Agile Aquatic Research Vessel; 2019. Available from: <https://clearpathrobotics.com>.
- [34] Macenski S, Foote T, Gerkey B, Lalancette C, Woodall W. Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*. 2022 May;7(66). Available from: <http://dx.doi.org/10.1126/scirobotics.abm6074>.
- [35] Koenig N, Howard A. Design and Use Paradigms for Gazebo, An Open-Source Multi-Robot Simulator. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems*. Sendai, Japan; 2004. p. 2149-54.
- [36] Agüero CE, Koenig N, Chen I, Boyer H, Peters S, Hsu J, et al. Inside the Virtual Robotics Challenge: Simulating Real-Time Robotic Disaster Response. *Automation Science and Engineering, IEEE Transactions on*. 2015 April;12(2):494-506.
- [37] Todorov E, Erez T, Tassa Y. MuJoCo: A physics engine for model-based control. In: *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems. IEEE*; 2012. p. 5026-33.